

§ 5.3 二叉树的实现

作为图的特殊形式，二叉树的基本组成单元是节点与边；作为数据结构，其基本的组成实体是二叉树节点（binary tree node），而边则对应于节点之间的相互引用。

5.3.1 二叉树节点

■ BinNode模板类

以二叉树节点BinNode模板类，可定义如代码5.1所示。

```

1 #define BinNodePosi(T) BinNode<T>* //节点位置
2 #define stature(p) ((p) ? (p)->height : -1) //节点高度（与“空树高度为-1”的约定相统一）
3 typedef enum { RB_RED, RB_BLACK } RBColor; //节点颜色
4
5 template <typename T> struct BinNode { //二叉树节点模板类
6 // 成员（为简化描述起见统一开放，读者可根据需要进一步封装）
7     T data; //数值
8     BinNodePosi(T) parent; BinNodePosi(T) lChild; BinNodePosi(T) rChild; //父节点及左、右孩子
9     int height; //高度（通用）
10    int npl; //Null Path Length（左式堆，也可直接用height代替）
11    RBColor color; //颜色（红黑树）
12 // 构造函数
13    BinNode() : parent(NULL), lChild(NULL), rChild(NULL), height(0), npl(1), color(RB_RED) { }
14    BinNode(T e, BinNodePosi(T) p = NULL, BinNodePosi(T) lc = NULL, BinNodePosi(T) rc = NULL,
15            int h = 0, int l = 1, RBColor c = RB_RED)
16        : data(e), parent(p), lChild(lc), rChild(rc), height(h), npl(l), color(c) { }
17 // 操作接口
18    int size(); //统计当前节点后代总数，亦即以其为根的子树的规模
19    BinNodePosi(T) insertAsLC(T const&); //作为当前节点的左孩子插入新节点
20    BinNodePosi(T) insertAsRC(T const&); //作为当前节点的右孩子插入新节点
21    BinNodePosi(T) succ(); //取当前节点的直接后继
22    template <typename VST> void travLevel(VST&); //子树层次遍历
23    template <typename VST> void travPre(VST&); //子树先序遍历
24    template <typename VST> void travIn(VST&); //子树中序遍历
25    template <typename VST> void travPost(VST&); //子树后序遍历
26 // 比较器、判等器（各列其一，其余自行补充）
27    bool operator<(BinNode const& bn) { return data < bn.data; } //小于
28    bool operator==(BinNode const& bn) { return data == bn.data; } //等于
29 };

```

代码5.1 二叉树节点模板类BinNode

为简化起见，这里也未做严格的封装。通过宏BinNodePosi来指代节点位置，可以简化后续代码描述；通过定义宏stature，则可以保证从节点返回的高度值，能够与“空树高度为-1”的约定相统一。

■ 成员变量

如图5.10所示, `BinNode`节点由多个成员变量组成, 它们分别记录了当前节点的父亲和孩子的位置、节点内存放的数据以及节点的高度等指标, 这些都是二叉树相关算法赖以实现的基础。

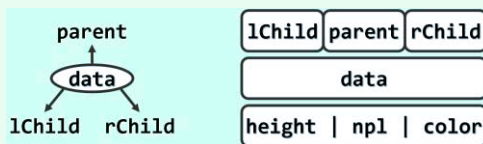


图5.10 `BinNode`模板类的逻辑结构

其中, `data`的类型由模板变量`T`指定, 用于存放各节点所对应的数值对象。`lChild`、`rChild`和`parent`均为指针类型, 分别指向左、右孩子以及父节点的位置。如此, 既可将各节点联接起来, 也可在它们之间漫游移动。比如稍后5.4节将要介绍的遍历算法, 就必须借助此类位置变量。当然, 通过判断这些变量所指位置是否为`NULL`, 也可确定当前节点的类型。比如, `v.parent = NULL`当且仅当`v`是根节点, 而`v.lChild = v.rChild = NULL`当且仅当`v`是叶节点。

后续章节将基于二叉树实现二叉搜索树和优先级队列等数据结构, 而节点高度`height`在其中的具体语义也有所不同。比如, 8.3节的红黑树将采用所谓的黑高度 (`black height`), 而10.3节的左式堆则采用所谓的空节点通路长度 (`null path length`)。尽管后者也可以直接沿用`height`变量, 但出于可读性的考虑, 这里还是专门设置了一个变量`npl`。

有些种类的二叉树还可能需要其它的变量来描述节点状态, 比如针对其中节点的颜色, 红黑树需要引入一个属于枚举类型`RB_Color`的变量`color`。

根据不同应用需求, 还可以针对节点的深度增设成员变量`depth`, 或者针对以当前节点为根的子树规模 (该节点的后代数) 增设成员变量`size`。利用这些变量固然可以加速静态的查询或搜索, 但为保持这些变量的时效性, 在所属二叉树发生结构性调整 (比如节点的插入或删除) 之后, 这些成员变量都要动态地更新。因此, 究竟是否值得引入此类成员变量, 必须权衡利弊。比如, 在二叉树结构改变频繁以至于动态操作远多于静态操作的场合, 舍弃深度、子树规模等变量, 转而在实际需要时再直接计算这些指标, 应是更为明智的选择。

■ 快捷方式

在`BinNode`模板类各接口以及后续相关算法的实现中, 将频繁检查和判断二叉树节点的状态与性质, 有时还需要定位与之相关的 (兄弟、叔叔等) 特定节点, 为简化算法描述同时增强可读性, 不妨如代码5.2所示将其中常用功能以宏的形式加以整理归纳。

```
1 /*****
2  * BinNode状态与性质的判断
3  *****/
4 #define IsRoot(x) (!(x).parent)
5 #define IsLChild(x) (!IsRoot(x) && (&(x) == (x).parent->lChild))
6 #define IsRChild(x) (!IsRoot(x) && (&(x) == (x).parent->rChild))
7 #define HasParent(x) (!IsRoot(x))
8 #define HasLChild(x) ((x).lChild)
9 #define HasRChild(x) ((x).rChild)
10 #define HasChild(x) (HasLChild(x) || HasRChild(x)) //至少拥有一个孩子
11 #define HasBothChild(x) (HasLChild(x) && HasRChild(x)) //同时拥有两个孩子
12 #define IsLeaf(x) (!HasChild(x))
```

```

13
14 /*****
15  * 与BinNode具有特定关系的节点及指针
16  *****/
17 #define sibling(p) ( \
18     IsLChild(*(p)) ? \
19         (p)->parent->rChild : \
20         (p)->parent->lChild \
21 ) //兄弟
22
23 #define uncle(x) ( \
24     IsLChild(*(x->parent)) ? \
25         (x)->parent->parent->rChild : \
26         (x)->parent->parent->lChild \
27 ) //叔叔
28
29 #define FromParentTo(x) ( \
30     IsRoot(x) ? _root : ( \
31     IsLChild(x) ? (x).parent->lChild : (x).parent->rChild \
32     ) \
33 ) //来自父亲的指针

```

代码5.2 以宏的形式对基于BinNode的操作做一归纳整理

5.3.2 二叉树节点操作接口

由于BinNode模板类本身处于底层，故这里也将所有操作接口统一设置为开放权限，以简化描述。同样地，注重数据结构封装性的读者可在此基础上自行修改扩充。

■ 插入孩子节点

```

1 template <typename T> //将e作为当前节点的左孩子插入二叉树
2 BinNodePosi(T) BinNode<T>::insertAsLC(T const& e) { return lChild = new BinNode(e, this); }
3
4 template <typename T> //将e作为当前节点的右孩子插入二叉树
5 BinNodePosi(T) BinNode<T>::insertAsRC(T const& e) { return rChild = new BinNode(e, this); }

```

代码5.3 二叉树节点左、右孩子的插入

可见，为将新节点作为当前节点的左孩子插入树中，可如图5.11(a)所示，先创建新节点；再如图(b)所示，将当前节点作为新节点的父亲，并令新节点作为当前节点的左孩子。这里约定，在插入新节点之前，当前节点尚无左孩子。

右孩子的插入过程完全对称，不再赘述。

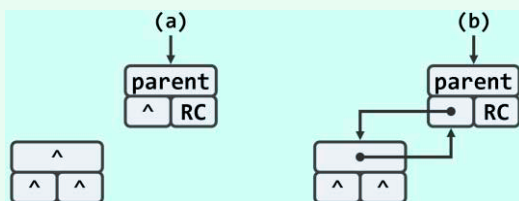


图5.11 二叉树节点左孩子插入过程

■ 定位直接后继

稍后在5.4.3节我们将会看到,通过中序遍历,可在二叉树各节点之间定义一个线性次序。相应地,各节点之间也可定义前驱与后继关系。这里的succ()接口,可以返回当前节点的直接后继(如果存在)。该接口的具体实现,将在130页代码5.16中给出。

■ 遍历

稍后的5.4节,将从递归和迭代两个角度,分别介绍各种遍历算法的不同实现。为便于测试与比较,不妨将这些算法的不同版本统一归入统一的接口中,并在调用时随机选择。

```
1 template <typename T> template <typename VST> //元素类型、操作器
2 void BinNode<T>::travIn(VST& visit) { //二叉树中序遍历算法统一入口
3     switch (rand() % 5) { //此处暂随机选择以做测试,共五种选择
4         case 1: travIn_I1(this, visit); break; //迭代版#1
5         case 2: travIn_I2(this, visit); break; //迭代版#2
6         case 3: travIn_I3(this, visit); break; //迭代版#3
7         case 4: travIn_I4(this, visit); break; //迭代版#4
8         default: travIn_R(this, visit); break; //递归版
9     }
10 }
```

代码5.4 二叉树中序遍历算法的统一入口

比如,中序遍历算法的五种实现方式(其中travIn_I4留作习题[5-17]),即可如代码5.4所示,纳入统一的BinNode::travIn()接口。其余遍历算法的处理方法类似,不再赘述。

5.3.3 二叉树

■ BinTree模板类

在BinNode模板类的基础之上,可如代码5.5所示定义二叉树BinTree模板类。

```
1 #include "BinNode.h" //引入二叉树节点类
2 template <typename T> class BinTree { //二叉树模板类
3 protected:
4     int _size; //规模
5     BinNodePosi(T) _root; //根节点
6     virtual int updateHeight(BinNodePosi(T) x); //更新节点x的高度
7     void updateHeightAbove(BinNodePosi(T) x); //更新节点x及其祖先的高度
8 public:
9     BinTree() : _size(0), _root(NULL) { } //构造函数
10    ~BinTree() { if (0 < _size) remove(_root); } //析构函数
11    int size() const { return _size; } //规模
12    bool empty() const { return !_root; } //判空
13    BinNodePosi(T) root() const { return _root; } //树根
14    BinNodePosi(T) insertAsRoot(T const& e); //插入根节点
15    BinNodePosi(T) insertAsLC(BinNodePosi(T) x, T const& e); //e作为x的左孩子(原无)插入
```

```

16  BinNodePosi(T) insertAsRC(BinNodePosi(T) x, T const& e); //e作为x的右孩子(原无)插入
17  BinNodePosi(T) attachAsLC(BinNodePosi(T) x, BinTree<T>* &T); //T作为x左子树接入
18  BinNodePosi(T) attachAsRC(BinNodePosi(T) x, BinTree<T>* &T); //T作为x右子树接入
19  int remove(BinNodePosi(T) x); //删除以位置x处节点为根的子树,返回该子树原先的规模
20  BinTree<T>* secede(BinNodePosi(T) x); //将子树x从当前树中摘除,并将其转换为一棵独立子树
21  template <typename VST> //操作器
22  void travLevel(VST& visit) { if (_root) _root->travLevel(visit); } //层次遍历
23  template <typename VST> //操作器
24  void travPre(VST& visit) { if (_root) _root->travPre(visit); } //先序遍历
25  template <typename VST> //操作器
26  void travIn(VST& visit) { if (_root) _root->travIn(visit); } //中序遍历
27  template <typename VST> //操作器
28  void travPost(VST& visit) { if (_root) _root->travPost(visit); } //后序遍历
29  // 比较器、判等器(各列其一,其余自行补充)
30  bool operator<(BinTree<T> const& t) { return _root && t._root && lt(_root, t._root); }
31  bool operator==(BinTree<T> const& t) { return _root && t._root && (_root == t._root); }
32 }; //BinTree

```

代码5.5 二叉树模板类BinTree

其中, `_root`指向树根, `_size`动态记录树的规模, 且 `_root = NULL` 当且仅当 `_size = 0`。

■ 高度更新

二叉树任一节点的高度, 都等于其孩子节点的最大高度加一。于是, 每当某一节点的孩子或后代有所增减, 其高度都有必要及时更新。然而实际上, 节点自身很难发现后代的变化, 因此这里反过来采用另一处理策略: 一旦有节点加入或离开二叉树, 则更新其所有祖先的高度。请读者自行验证, 这一原则实际上与前一个等效(习题[5-3])。

在每一节点 `v` 处, 只需读出其左、右孩子的高度并取二者之间的大者, 再计入当前节点本身, 就得到了 `v` 的新高度。通常, 接下来还需要从 `v` 出发沿 `parent` 指针逆行向上, 依次更新各代祖先的高度记录。这一过程可具体实现如代码5.6所示。

```

1  template <typename T> int BinTree<T>::updateHeight(BinNodePosi(T) x) //更新节点x高度
2  { return x->height = 1 + max(stature(x->lChild), stature(x->rChild)); } //具体规则因树不同而异
3
4  template <typename T> void BinTree<T>::updateHeightAbove(BinNodePosi(T) x) //更新v及祖先的高度
5  { while (x) { updateHeight(x); x = x->parent; } } //可优化: 一旦高度未变, 即可终止

```

代码5.6 二叉树节点的高度更新

更新每一节点本身的高度, 只需执行两次 `getHeight()` 操作、两次加法以及两次取最大操作, 不过常数时间, 故 `updateHeight()` 算法总体运行时间也是 $O(\text{depth}(v) + 1)$, 其中 `depth(v)` 为节点 `v` 的深度。当然, 这一算法还可进一步优化(习题[5-4])。

在某些种类的二叉树(例如8.3节将要介绍的红黑树)中, 高度的定义有所不同, 因此这里将 `updateHeight()` 定义为保护级的虚方法, 以便派生类在必要时重写(`override`)。

■ 节点插入

二叉树节点可以通过三种方式插入二叉树中，具体实现如代码5.7所示。

```
1 template <typename T> BinNodePosi(T) BinTree<T>::insertAsRoot(T const& e)
2 { _size = 1; return _root = new BinNode<T>(e); } //将e当作根节点插入空的二叉树
3
4 template <typename T> BinNodePosi(T) BinTree<T>::insertAsLC(BinNodePosi(T) x, T const& e)
5 { _size++; x->insertAsLC(e); updateHeightAbove(x); return x->lChild; } //e插入为x的左孩子
6
7 template <typename T> BinNodePosi(T) BinTree<T>::insertAsRC(BinNodePosi(T) x, T const& e)
8 { _size++; x->insertAsRC(e); updateHeightAbove(x); return x->rChild; } //e插入为x的右孩子
```

代码5.7 二叉树根、左、右节点的插入

`insertAsRoot()`接口用于将节点插入空树中，当然，该节点随即也应成为树根。

为此，只需创建一个新节点并存入指定的数据项，再令其作为根节点，同时更新全树的规模。

如图5.12(a)所示，若二叉树T中某个节点x的右孩子为空，则可通过`T.insertAsRC()`接口为其添加一个右孩子。为此可如图(b)所示调用`x->insertAsRC()`接口，将二者按照父子关系相互联接，同时通过`updateHeightAbove()`接口更新x所有祖先的高度，并更新全树规模。请注意这里的两个同名`insertAsRC()`接口，它们各自所属的对象类型不同。

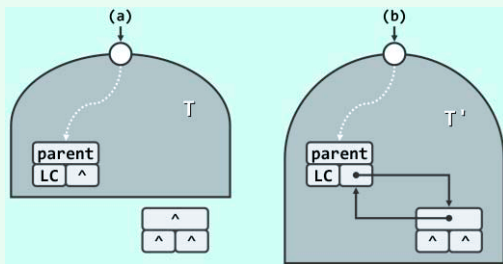


图5.12 右节点插入过程：(a)插入前；(b)插入后

左侧节点的插入过程与此相仿，可对称地调用`insertAsLC()`完成。

■ 子树接入

如代码5.8所示，任一二叉树均可作为另一二叉树中指定节点的左子树或右子树，植入其中。

```
1 template <typename T> //二叉树子树接入算法：将S当作节点x的左子树接入，S本身置空
2 BinNodePosi(T) BinTree<T>::attachAsLC(BinNodePosi(T) x, BinTree<T>* &S) { //x->lChild == NULL
3     if (x->lChild = S->_root) x->lChild->parent = x; //接入
4     _size += S->_size; updateHeightAbove(x); //更新全树规模与x所有祖先的高度
5     S->_root = NULL; S->_size = 0; release(S); S = NULL; return x; //释放原树，返回接入位置
6 }
7
8 template <typename T> //二叉树子树接入算法：将S当作节点x的右子树接入，S本身置空
9 BinNodePosi(T) BinTree<T>::attachAsRC(BinNodePosi(T) x, BinTree<T>* &S) { //x->rChild == NULL
10    if (x->rChild = S->_root) x->rChild->parent = x; //接入
11    _size += S->_size; updateHeightAbove(x); //更新全树规模与x所有祖先的高度
12    S->_root = NULL; S->_size = 0; release(S); S = NULL; return x; //释放原树，返回接入位置
13 }
```

代码5.8 二叉树子树的接入

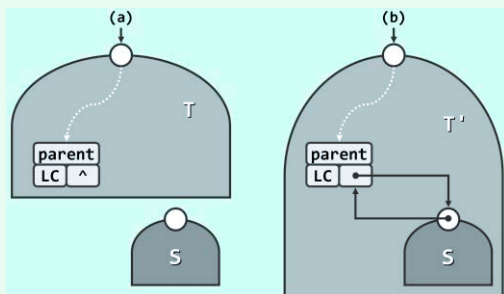


图5.13 右子树接入过程：(a)接入前；(b)接入后

如图5.13(a)，若二叉树T中节点x的右孩子为空，则attachAsRC()接口首先将待植入的二叉树S的根节点作为x的右孩子，同时令x作为该根节点的父亲；然后，更新全树规模以及节点x所有祖先的高度；最后，将树S中除已接入的各节点之外的其余部分归还系统。

左子树接入过程与此类似，可对称地调用attachAsLC()完成。

■ 子树删除

子树删除的过程与如图5.13所示的子树接入过程恰好相反，不同之处在于，需要将被摘除子树中的节点逐一释放并归还系统（习题[5-5]）。具体实现如代码5.9所示。

```

1 template <typename T> //删除二叉树中位置x处的节点及其后代，返回被删除节点的数值
2 int BinTree<T>::remove(BinNodePosi(T) x) { //assert: x为二叉树中的合法位置
3     FromParentTo(*x) = NULL; //切断来自父节点的指针
4     updateHeightAbove(x->parent); //更新祖先高度
5     int n = removeAt(x); _size -= n; return n; //删除子树x，更新规模，返回删除节点总数
6 }
7
8 template <typename T> //删除二叉树中位置x处的节点及其后代，返回被删除节点的数值
9 static int removeAt(BinNodePosi(T) x) { //assert: x为二叉树中的合法位置
10    if (!x) return 0; //递归基：空树
11    int n = 1 + removeAt(x->lChild) + removeAt(x->rChild); //递归释放左、右子树
12    release(x->data); release(x); return n; //释放被摘除节点，并返回删除节点总数
13 }

```

代码5.9 二叉树子树的删除

■ 子树分离

子树分离的过程与以上的子树删除过程基本一致，不同之处在于，需要对分离出来的子树重新封装，并返回给上层调用者。具体实现如代码5.10所示。

```

1 template <typename T> //二叉树子树分离算法：将子树x从当前树中摘除，将其封装为一棵独立子树返回
2 BinTree<T>* BinTree<T>::secede(BinNodePosi(T) x) { //assert: x为二叉树中的合法位置
3     FromParentTo(*x) = NULL; //切断来自父节点的指针
4     updateHeightAbove(x->parent); //更新原树中所有祖先的高度
5     BinTree<T>* S = new BinTree<T>; S->_root = x; x->parent = NULL; //新树以x为根
6     S->_size = x->size(); _size -= S->_size; return S; //更新规模，返回分离出来的子树
7 }

```

代码5.10 二叉树子树的分离

■ 复杂度

就二叉树拓扑结构的变化范围而言，以上算法均只涉及局部的常数个节点。因此，除了更新祖先高度和释放节点等操作，只需常数时间。

§ 5.4 遍历

对二叉树的访问多可抽象为如下形式：按照事先约定的某种规则或次序，对节点各访问一次而且仅一次。与向量和列表等线性结构一样，二叉树的这类访问也统称为遍历（traversal）。同样地，遍历操作之于二叉树的意义，在于为许多相关算法的实现提供了通用框架和基本接口。从算法策略的角度看，这一过程也等效于将半线性的树形结构转换为线性结构。

不过，因为二叉树已经不再属于线性结构，故相对于向量和列表等序列结构，二叉树的遍历略显复杂。为此，以下首先从递归的角度，给出若干种典型的二叉树遍历次序的定义，并按照117页代码5.1和121页代码5.5所列接口，给出相应的递归式实现；然后，为了提高遍历算法的实际效率，再分别介绍各种遍历接口的迭代式实现。

5.4.1 递归式遍历

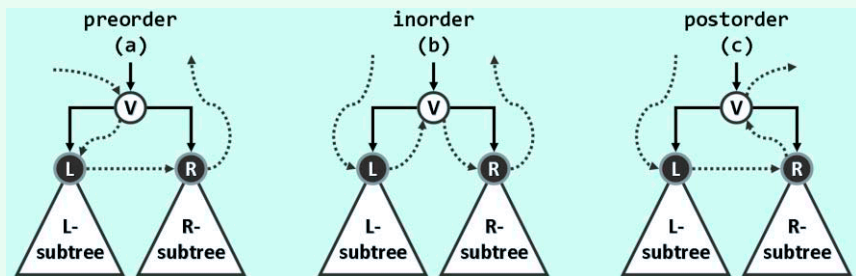


图5.14 二叉树遍历的全局次序由局部次序规则确定

二叉树本身并不具有天然的全局次序，故为实现遍历，需通过在各节点与其孩子之间约定某种局部次序，间接地定义某种全局次序。

按惯例左兄弟优先于右兄弟，故若将节点及其孩子分别记作V、L和R，则如图5.14所示，局部访问的次序可有VLR、LVR和LRV三种选择。根据节点V在其中的访问次序，三种策略也相应地分别称作先序遍历、中序遍历和后序遍历，分述如下。

■ 先序遍历

得益于递归定义的简洁性，如代码5.11所示，只需数行即可实现先序遍历算法。

```
1 template <typename T, typename VST> //元素类型、操作器
2 void travPre_R(BinNodePosi(T) x, VST& visit) { //二叉树先序遍历算法（递归版）
3     if (!x) return;
4     visit(x->data);
5     travPre_R(x->lChild, visit);
6     travPre_R(x->rChild, visit);
7 }
```

代码5.11 二叉树先序遍历算法（递归版）

为遍历（子）树x，首先核对x是否为空。若x为空，则直接退出——其效果相当于递归基。反之，若x非空，则按照先序遍历关于局部次序的定义，优先访问其根节点x；然后，依次深入左子树和右子树，递归地进行遍历。实际上，这一实现模式也同样可以应用于中序和后序遍历。